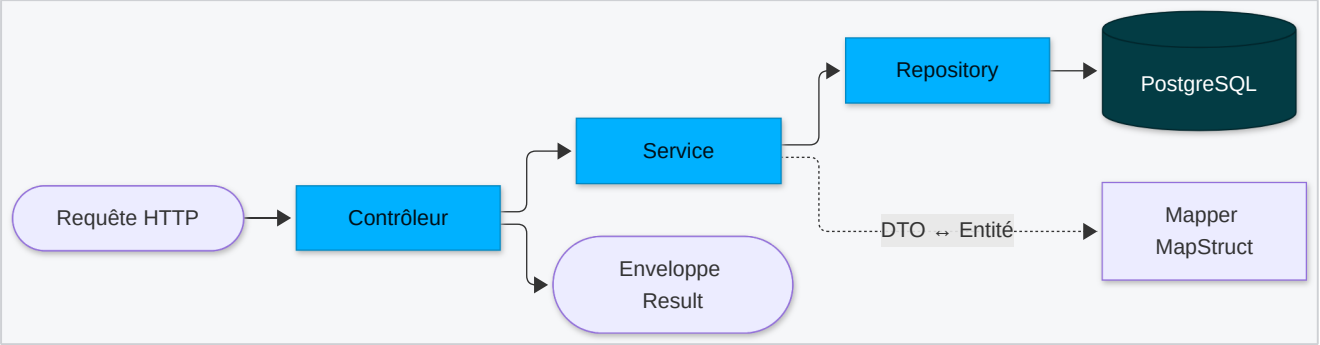
Back-end

`candex-api` est une application Spring Boot en couches. Ce handbook couvre comment elle est construite ; pour le domaine qu'elle sert, voir Base de données & Modèle, et pour le système autour, Architecture & Vue d'ensemble.

Flux d'une requête

Chaque requête traverse les mêmes couches. Les contrôleurs renvoient une enveloppe typée **`Result`** ; les services portent la logique ; les repositories sont du Spring Data JPA ; et la frontière entre entités JPA et formes de l'API est franchie par des mappers **MapStruct**.



Chaque couche a exactement un rôle :

Couche	Responsabilité	Ne doit pas
Contrôleur	Associer une requête HTTP à une méthode, contrôler l'accès (`@PreAuthorize`), renvoyer un `Result`. Mince.	Porter de la logique métier ni toucher la base
Service (`service/core`)	La logique métier : orchestration, transactions, vérification des règles, appel des repositories, mapping entités ↔ DTO, levée d'exceptions.	Connaître quoi que ce soit du HTTP
Repository	Accès Spring Data JPA à un agrégat — requêtes dérivées et personnalisées.	Porter des règles métier
Entité	Le modèle de domaine JPA (état persisté).	Franchir la frontière du service
DTO (`dto/request`, `dto/response`)	Les formes d'entrée et de sortie de l'API.	Porter des préoccupations de persistance
Mapper (MapStruct)	Convertir entités ↔ DTO à la frontière du service.	—
Transverse (`security`, `exception`, `validation`, `config`, `job`, `result`)	Auth, erreurs structurées, validation, configuration, jobs planifiés, enveloppe de réponse.	—

La règle d'or : les entités n'atteignent jamais le contrôleur, et les contrôleurs ne portent jamais de logique.

L'enveloppe de réponse

Chaque endpoint renvoie un `Result<T>` générique :

```
{ "success": true, "data": { "...": "..." }, "error": null }
```

- `data` porte la charge utile en cas de succès ; `error` porte une erreur structurée en cas d'échec ; `success` est le drapeau.
- Les réponses de collection ajoutent un en-tête **`X-Total-Count`** ; les créations ajoutent un en-tête **`Location`**.

Surface de l'API

Les chemins des ressources sont centralisés dans `ApiPaths` et exposés sous la base de l'API (par exemple `/api/drinks`). Environ **45 endpoints** répartis sur ~14 ressources (28 `GET`, 9 `POST`, 4 `DELETE`, 3 `PUT`, 1 `PATCH`) :

Ressource	Chemin
Drinks	`/drinks` (dont import en masse `import`, `import/diff`, `import/sync`)
Cans • Can formats	`/cans` • `/can-formats`
Brands	`/brands`
Flavors • Flavor categories	`/flavors` • `/flavor-categories`
Colors • Drink categories	`/colors` • `/drink-categories`
World Dex	`/world-dex`
Users • Unlocked drinks	`/users` • `/user-unlocked-drinks`
App versions • Reports	`/app-versions` • `/reports`
Console SQL admin	`/admin/...`

La doc interactive est sur **Swagger UI** (`/swagger-ui.html`). Les contrats exacts requête/réponse relèvent du handbook **API / Contrats** *(à venir)*.

Autorisation

Les endpoints sont gardés par `@PreAuthorize` et `SecurityExpressions` :

Règle	Pour
`USER_OR_ADMIN`	Lectures authentifiées (catalogue, World Dex)
`ADMIN_ONLY`	Écritures du catalogue et opérations admin
`PERMIT_ALL`	Les rares endpoints publics
`@securityAccess.canAccessUserId(...)`	Données propres à l'utilisateur — chacun ne touche que ses propres enregistrements

L'identité vient de Firebase (voir Architecture & Vue d'ensemble).

Services & logique métier

La logique métier est dans `service/core` — un service par ressource (`DrinkService`, `BrandService`, `WorldDexService`, `UserService`, …) — plus des services de support : `ImageService` (analyse), `UploaderService` (média), `ReportService` et `SqlConsoleService`.

Gestion d'erreurs

Chaque exception étend un `CustomException` commun, qui **porte son propre `HttpStatus`** et se sérialise dans le champ `error` de l'enveloppe `Result` (un `dto/response/Error` contenant le message et le code de statut). Les exceptions se répartissent en deux familles.

Erreurs API (`exception/api`) — problèmes de requête et de données :

Exception	HTTP	Levée quand
`ElementNotFoundException`	404	Une entité référencée n'existe pas
`InvalidFieldValueException`	400	Un champ a un format ou une valeur invalide
`MissingRequiredFieldException`	400	Un champ requis est absent
`UnsupportedFieldException`	400	Un champ inconnu/non supporté est utilisé
`InvalidImportPayloadException`	400	Un payload d'import en masse est malformé
`SqlExecutionException`	400	Une requête de la console SQL échoue
`ImmutableFieldModificationException`	409	Un champ immuable est modifié après création
`UniqueConstraintViolationException`	409	Une contrainte d'unicité est violée
`MissingConfigurationException`	422	Une configuration requise (ex. une version d'app) manque

Erreurs métier (`exception/business`) — violations de règles de domaine :

Exception	HTTP	Levée quand
`MissingRequiredCanException`	400	Une boisson est enregistrée sans aucune canette
`MissingRequiredLocalizationException`	400	Une boisson est enregistrée sans aucune localisation
`InvalidDexPositionException`	400	Une position du World Dex est hors limites
`InvalidEditionConfigurationException`	400	Une édition / variante est mal configurée
`DrinkAlreadyHasPredecessorException`	409	Une boisson est liée comme prédécesseur à deux boissons
`ImageAnalysisException`	422	L'analyse d'image échoue

En bref : les exceptions `api` décrivent ce qui ne va pas dans la requête ; les exceptions `business` décrivent quelle règle métier a été enfreinte.

Validation

Les DTO de requête sont validés avec Bean Validation, câblé via `ValidationConfig`. Une entrée invalide ressort en erreur `api` plutôt que d'atteindre la couche service.

Jobs planifiés

Deux jobs de fond tournent périodiquement :

- `UnverifiedUserCleanupJob` — supprime les utilisateurs n'ayant jamais vérifié leur compte.
- `WorldDexHistoryPruneJob` — élague l'ancien historique du World Dex.

Configuration

La configuration vit dans `config` et dépend du profil (`dev` / `prod`) : `OpenApiConfig` (Swagger), `CorsConfig` (origines autorisées), `JacksonConfig` (JSON), `SqlConsoleConfig` et `ValidationConfig`. Pour la lancer en local, voir Installation locale ; les spécificités d'environnement relèvent du handbook **Déploiement** *(à venir)*.

Conventions

- Les contrôleurs renvoient `ResponseEntity<Result<T>>` ; les chemins viennent des constantes `ApiPaths`.
- Les entités ne quittent jamais la frontière du service — les DTO (`dto/request`, `dto/response`) la franchissent via les mappers MapStruct.
- Chaque endpoint déclare son accès avec `@PreAuthorize`.
- Les erreurs sont des exceptions structurées (`api` / `business`) exposées via l'enveloppe `Result`.